



Guía de Organización del Repositorio en GitHub

Esta guía establece la estructura y las reglas de colaboración para el proyecto final del curso. Un repositorio bien organizado facilita la revisión y el despliegue final en local o en la nube (AWS).

1. Estructura de Carpetas Sugerida

Dependiendo del framework elegido (**Django** o **Flask**), la estructura debe ser modular.

Para proyectos en Django:

```
/
├── core/          # Configuración principal (settings.py, urls.py)
├── apps/          # Módulos: terceros, inventario, facturación
├── static/        # Archivos CSS, JS e imágenes
├── templates/     # Archivos HTML
├── db/           # Scripts SQL (DDL, Vistas, Triggers)
├── docs/         # Diagramas ER y documentación adicional
├── .gitignore     # Archivos omitidos (venv, .env, __pycache__)
├── manage.py      # Script de ejecución
└── requirements.txt # Dependencias del proyecto
```

Para proyectos en Flask/FastAPI:

```
/
├── app/          # Código fuente de la aplicación
│   ├── routes/   # Endpoints del sistema
│   ├── models/   # Modelos de base de datos
│   └── services/  # Lógica de negocio (Cálculos de stock, IVA)
├── database/     # Scripts SQL (DDL, Vistas, Triggers)
├── static/       # Archivos CSS, JS e imágenes
├── templates/    # Archivos HTML
├── .gitignore
├── config.py     # Configuración de PostgreSQL
├── requirements.txt
└── run.py        # Punto de entrada de la app
```

2. Archivos Críticos de Configuración

El archivo .gitignore

Es **obligatorio** para no subir basura al repositorio. Debe incluir:

```
venv/  
*.pyc  
__pycache__/  
.env  
.DS_Store (para usuarios de Mac)
```

El archivo README.md

Es la cara del proyecto. Debe seguir la plantilla que definimos previamente, incluyendo el nombre de la empresa, integrantes y la guía de instalación local.

Bitácora de IA (DOC_IA.md)

Aquí deben documentar los prompts, resultados y ajustes manuales realizados con apoyo de LLMs. Es un requisito de transparencia y aprendizaje.

3. Reglas de Colaboración (Workflow)

Para grupos de 4 personas, trabajar todos en la rama main es arriesgado. Se recomienda el flujo **Feature Branch**:

1. **Main:** Solo contiene código estable que funciona.
 2. **Develop:** Rama para integrar las funcionalidades de todos los compañeros.
 3. **Features:** Cada integrante crea una rama para su tarea (ej: git checkout -b feature/registro-clientes).
 4. **Pull Requests (PR):** Al terminar una tarea, se solicita integrar a develop. Otro compañero debería revisar que el código no rompa la base de datos.
-

4. Gestión de la Base de Datos en GitHub

Como este es un curso de **Bases de Datos**, el código SQL es tan importante como el código Python. Los estudiantes deben:

- **Versionar el SQL:** Cada vez que cambien una tabla (ej: agreguen un campo de "Habeas Data"), deben actualizar el archivo .sql en la carpeta db/.
- **Datos de Prueba:** Incluir un script con al menos 5 insumos de ejemplo (como los del enunciado) para que el docente pueda probar la app rápidamente.

5. Recomendaciones Finales para el Estudiante

- **Commits descriptivos:** Evitar mensajes como "cambios" o "fix". Usar algo como: Feat: Agregar validación de NIT para empresas colombianas.
- **Uso de Issues:** Pueden usar la pestaña "Issues" de GitHub para repartirse las tareas (ej: "Crear Trigger de inventario").
- **Privacidad:** El repositorio puede ser **público**.